TOPIC-2 PANDAS:

1Q) What is Pandas?

Ans:

Pandas is a Python library used to organize, analyze, and work with data easily.

2Q) Explain fundamental data structures of Pandas.

Ans:

1. Series

A **Series** is a **one-dimensional labeled array** that can hold any data type (integers, strings, floats, Python objects, etc.).

Key Features:

- Has both **values** and **index**.
- Index is like labels for each element.
- Similar to a NumPy array, but with labels (indexes).

Example:

```
import pandas as pd

data = pd.Series([10, 20, 30, 40], index=['a', 'b', 'c', 'd'])

print(data)
```

Output:

a 10

b 20

c 30

d 40

dtype: int64

Attributes:

- data.values → The actual data as a NumPy array.
- data.index → The index (row labels).

2. DataFrame

A **DataFrame** is a **two-dimensional labeled data structure** with columns of potentially different types.

Think of it as:

- A dictionary of Series.
- A table or spreadsheet in memory.

Example:

data = {

'Name': ['Alice', 'Bob', 'Charlie'],

'Age': [25, 30, 35],

'City': ['NYC', 'LA', 'Chicago']

}

df = pd.DataFrame(data)

print(df)

Output:

```
      Name  Age       City
0    Alice   25      NYC
1      Bob   30       LA
2  Charlie   35  Chicago
```

Attributes and Methods:

- df.columns → List of column names.
- df.index → Row indices.
- df.values → 2D array of data.
- df.describe() → Summary statistics for numeric columns.
- df.head() / df.tail() → View first or last few rows.

3. Index

An **Index** object in Pandas is an immutable array-like object used for labeling axis.

Features:

- Supports label-based and position-based indexing.
- Can be strings, numbers, or dates.

- Used in both Series and DataFrame.

Example:

```
idx = pd.Index(['a', 'b', 'c', 'd'])
print(idx)
```

Output:

```
Index(['a', 'b', 'c', 'd'], dtype='object')
```

3Q) How to access data in Pandas.

Ans:

Data Indexing and Selection

Access Methods in series

Example

```
import pandas as pd
s = pd.Series([100, 200, 300], index=['a', 'b', 'c'])
print(s['a'])      # Label-based access
print(s[0])        # Position-based access
```

What does the Series s look like?

```
a    100
```

b 200

c 300

dtype: int64

Explanation of Access Methods

1. s['a'] → Label-based Access

- You're telling Pandas: **"Give me the value associated with the label 'a'."**
- Since 'a' is the index of the first element, the result is:

100

---Equivalent to accessing a dictionary by key, like my_dict['a'].

2. s[0] → Position-based Access

- You're telling Pandas: **"Give me the first element (position 0)."**
- Even though you assigned custom labels, **you can still use integer positions** like a list.
- The element at position 0 is still 100.

100

---Equivalent to accessing a list item by index, like my_list[0].

.loc[] and .iloc[] in Pandas

1 .loc[] — Label-based Selection

Definition:

.loc[] is used to access data using explicit labels for rows and columns.

Example:

```
import pandas as pd
df = pd.DataFrame({
        'Name': ['Alice', 'Bob', 'Charlie'],
        'Age': [24, 27, 22],
        'City': ['NYC', 'LA', 'Chicago']
}, index=['a', 'b', 'c'])

# Access row with label 'a'
print(df.loc['a'])

# Access value at row 'b', column 'Age'
print(df.loc['b', 'Age'])

# Access multiple rows and specific columns
```

```
print(df.loc[['a', 'c'], ['Name', 'City']])
```

Line-by-line Explanation:

df.loc['a']

Returns the entire row labeled 'a':

```
Name Alice
Age          24
City   NYC
Name: a, dtype: object
```

df.loc['b', 'Age']

Returns the Age of the person in row 'b':

```
27
```

df.loc[['a', 'c'], ['Name', 'City']]

Selects rows 'a' and 'c', and columns 'Name' and 'City':

```
      Name    City
```

a Alice NYC

c Charlie Chicago

2 .iloc[] — Position-based Selection

Definition:

.iloc[] is used to access data using integer positions (like Python lists or arrays).

Example:

```
import pandas as pd
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [24, 27, 22],
    'City': ['NYC', 'LA', 'Chicago']
}, index=['a', 'b', 'c'])


# Access first row (position 0)
print(df.iloc[0])
```

```
# Access value at row 1, column 1
print(df.iloc[1, 1])

# Access first two rows, and first two columns
print(df.iloc[0:2, 0:2])
```

Line-by-line Explanation:

df.iloc[0]

Returns the first row (position 0):

```
Name Alice
Age          24
City   NYC
Name: a, dtype: object
```

Even though the row is labeled 'a', .iloc[0] means “first row.”

df.iloc[1, 1]

Returns the value at row index 1 and column index 1:

- Row 1 → 'b'
- Column 1 → 'Age'
- So output is:

27

df.iloc[0:2, 0:2]

Returns a subset of rows 0 to 1 (excluding 2) and columns 0 to 1 (excluding 2):

	Name	Age
a	Alice	24
b	Bob	27

Output

	Name	Age	City
a	Alice	24	NYC
b	Bob	27	LA
c	Charlie	22	Chicago

4Q) Explain Condition based Data selection in Pandas.

Ans:

Condition-Based Data Selection in Pandas (Boolean Indexing)

Definition (1 line)

Condition-based data selection is the process of selecting rows that satisfy a given condition.

It is also called **Boolean Indexing** because the condition produces **True** or **False** values.

Syntax

df[condition]

or

df[df["Column_Name"] > value]

If the condition is **True**, the row is selected.
If the condition is **False**, the row is ignored.

Example DataFrame

```
import pandas as pd

data = {
    "Name": ["Ravi", "Priya", "Kiran", "Anu"],
    "Age": [18, 19, 18, 20],
    "Marks": [90, 85, 95, 70]
}

df = pd.DataFrame(data)
print(df)
```

Output

Index	Name	Age	Marks
0	Ravi	18	90
1	Priya	19	85
2	Kiran	18	95
3	Anu	20	70

How it Works

Suppose we write:

df["Marks"] > 90

Pandas checks every value in the **Marks** column.

Marks	Condition (>90)
90	False
85	False
95	True
70	False

Output

0 False

1 False

2 True

3 False

Name: Marks, dtype: bool

Now if we write

df[df["Marks"] > 90]

Pandas keeps only the rows where the condition is **True**.

Output

Name	Age	Marks
Kiran	18	95

Examples

1. Select students whose marks are greater than 80

df[df["Marks"] > 80]

Output

Name	Age	Marks

Ravi	18	90
Priya	19	85
Kiran	18	95

2. Select students whose age is 18

df[df["Age"] == 18]

Output

Name	Age	Marks
Ravi	18	90
Kiran	18	95

3. Select students whose marks are less than 90

df[df["Marks"] < 90]

Output

Name	Age	Marks
Priya	19	85
Anu	20	70

4. Select students whose age is not 18

df[df["Age"] != 18]

Output

Name	Age	Marks
Priya	19	
Anu	20	

Multiple Conditions

Use **& (AND)**, **| (OR)**, and **~ (NOT)**.

AND (&)

Select students whose age is 18 **and** marks are greater than 90.

```
df[(df["Age"] == 18) & (df["Marks"] > 90)]
```

Output

Name	Age	Marks
Kiran	18	95

OR (|)

Select students whose age is 20 **or** marks are greater than 90.

```
df[(df["Age"] == 20) | (df["Marks"] > 90)]
```

Output

Name	Age	Marks

Kiran	18	95
Anu	20	70

NOT (~)

Select students whose age is **not** 18.

df[~(df["Age"] == 18)]

Output

Name	Age	Marks
Priya	19	85
Anu	20	70

Comparison Operators Used

Operator	Meaning	Example

`==`	Equal to	`df["Age"] == 18`
`!=`	Not equal to	`df["Age"] != 18`
`>`	Greater than	`df["Marks"] > 90`
`<`	Less than	`df["Marks"] < 90`
`>=`	Greater than or equal to	`df["Marks"] >= 90`
`<=`	Less than or equal to	`df["Marks"] <= 90`

Important Points

- The condition returns a **Boolean Series** (`True` or `False`).
- Only rows with **True** are selected.
- Use `&` for **AND**, `|` for **OR**, and `~` for **NOT**.
- **Always put each condition inside parentheses** when using `&` or `|`.

Exam Definition (2 marks)

Condition-based data selection is the process of selecting rows from a Pandas DataFrame that satisfy a specified condition using Boolean expressions.

5Q) Explain Operations on Data in Pandas.

Ans:

Definition:

Operating on data refers to applying mathematical, comparison, logical, and transformation functions to Series or DataFrame objects in Pandas.

Operations can be:

- Element-wise (like NumPy)
- Broadcasted across rows or columns
- Vectorized for speed and performance

1. Arithmetic Operations

You can perform arithmetic operations **directly** on Series and DataFrames.

▸ **Example:**

```
import pandas as pd

# Create a DataFrame

df = pd.DataFrame({

    'A': [1, 2, 3],
```

```
    'B': [4, 5, 6]
})

# Add 10 to all elements
print(df + 10)

# Subtract column B from A
print(df['A'] - df['B'])

# Add two DataFrames
print(df + df)
```

Output

```
   A   B
0  2   8
1  4  10
2  6  12
```

Explanation:

-àdf + 10

Adds 10 to every element in the DataFrame:

```
    A   B
0  11  14
1  12  15
2  13  16
```

-à df['A'] - df['B']

Subtracts column 'B' from column 'A' **element-wise**:

```
0   -3
1   -3
2   -3
dtype: int64
```

à df + df

Adds the DataFrame to itself:

```
    A   B
0   2   8
1   4  10
2   6  12
```

2. Comparison Operations

Used for filtering or logical operations.

▸ **Example:**

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({
    'A': [1, 2, 3],
    'B': [4, 5, 6]
})

# Find where values in A > 1
print(df['A'] > 1)

# Filter rows where B > 4
print(df[df['B'] > 4])
```

Output

```
0    False
1    True
2    True
```

Name: A, dtype: bool

	A	B
1	2	5
2	3	6

Explanation:

->df['A'] > 1

Returns a Boolean Series:

0 False

1 True

2 True

Name: A, dtype: bool

àdf[df['B'] > 4]

Filters rows where 'B' > 4:

	A	B
1	2	5
2	3	6

3. Statistical Operations

Pandas supports built-in statistical methods.

▸ **Example:**

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({
        'A': [1, 2, 3],
        'B': [4, 5, 6]
})



# Mean of each column
print(df.mean())

# Max of each column
print(df.max())

# Sum of each row
print(df.sum(axis=1))
```

Output

A 2.0

B 5.0

dtype: float64

A 3

B 6

dtype: int64

0 5

1 7

2 9

dtype: int64

Explanation:

àdf.mean()

Returns the **mean** of each column:

A 2.0

B 5.0

dtype: float64

àdf.max()

Returns the **max** of each column:

A 3

B 6

dtype: int64

à **df.sum(axis=1)**

Returns the **row-wise sum** (axis=1 means row):

0 5

1 7

2 9

dtype: int64

6Q) How to handle missing data in Pandas.

Ans:

Handling Missing Data in Pandas (Simple English)

What is Missing Data?

Definition (1 line):
Missing data means some values are not available or are left empty in a dataset.

In real life, when we collect data (in Excel, CSV files, databases, etc.), some information may be missing.

Examples

Suppose you have the following student data:

Name	Age	City
Alice	25	NYC
Bob	*(empty)*	LA
(empty)	30	Chicago

Here:

- Bob's **Age** is missing.
- Third student's **Name** is missing.

These empty values are called **missing data**.

Why is Missing Data a Problem?

Missing data can create problems because it can affect:

- **Data Analysis** – Results may become incorrect.
- **Machine Learning Models** – The model may not learn properly.

- **Data Visualization** – Charts and graphs may be inaccurate.

How does Pandas represent missing data?

Pandas uses different symbols to represent missing values depending on the data type.

Data Type	Missing Value Representation
Numeric data (int, float)	`NaN` (Not a Number)
Object data (strings/text)	`None`

Example

Age = NaN

Name = None

Pandas treats both `NaN` and `None` as **missing values.**

Pandas provides built-in functions to:

- Detect missing values
- Remove missing values

- Fill missing values

1. Detecting Missing Data

Sometimes we first need to know **where the missing values are.**

Pandas provides four methods.

Method	Purpose
`isna()`	Returns `True` where data is missing
`isnull()`	Same as `isna()`
`notna()`	Returns `True` where data exists
`notnull()`	Same as `notna()`

Example

import pandas as pd

import numpy as np

```
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', None],
    'Age': [25, np.nan, 30],
    'City': ['NYC', 'LA', 'Chicago']
})

print(df.isna())
```

Original DataFrame

Name	Age	City
Alice	25	NYC
Bob	NaN	LA
None	30	Chicago

Output

	Name	Age	City
0	False	False	False
1	False	True	False
2	True	False	False

How to understand this?

Pandas checks **every cell.**

- If the value is present → `False`
- If the value is missing → `True`

Row-wise explanation

Row 0

- Name = Alice → available → `False`
- Age = 25 → available → `False`
- City = NYC → available → `False`

No missing values.

Row 1

- Name = Bob → available → `False`
- Age = NaN → missing → `True`

- City = LA → available → `False`

Only Age is missing.

Row 2

- Name = None → missing → `True`
- Age = 30 → available → `False`
- City = Chicago → available → `False`

Only Name is missing.

This helps us identify exactly **where the missing values are**.

2. Removing Missing Data

Sometimes we don't want rows or columns with missing values.

We can remove them using:

dropna()

Example 1: Remove rows with missing values

print(df.dropna())

Original DataFrame

Name	Age	City
Alice	25	NYC
Bob	NaN	LA
None	30	Chicago

Output

Name	Age	City
Alice	25	NYC

Explanation

Pandas checks each row.

- Row 0 → No missing value → Keep it.
- Row 1 → Missing Age → Remove it.

- Row 2 → Missing Name → Remove it.

Only Row 0 remains.

Example 2: Remove columns with missing values

print(df.dropna(axis=1))

Output

City
NYC
LA
Chica go

Explanation

Pandas checks every column.

- Name column → has one missing value → Remove it.
- Age column → has one missing value → Remove it.
- City column → no missing values → Keep it.

Only the **City** column remains.

3. Filling Missing Data

Instead of removing data, we can **replace the missing values.**

We use:

fillna()

We can fill missing values using:

- A single constant value
- Different values for different columns
- Previous row's value (Forward Fill)
- Next row's value (Backward Fill)

Example 1: Fill all missing values with one value

print(df.fillna("Unknown"))

Output

Name	Age	City
Alice	25	NYC

Bob	Unknown	LA
Unknown	30	Chicago

Explanation

Every missing value becomes **"Unknown"**.

- Missing Age → Unknown
- Missing Name → Unknown

Example 2: Fill different columns with different values

```
print(df.fillna({
    "Name":"No Name",
    "Age":0
}))
```

Output

Name	Age	City

Alice	25	NYC
Bob	0	LA
No Name	30	Chicago

Explanation

Different replacement values are used.

- Missing Name → "No Name"
- Missing Age → 0

Each column gets its own replacement value.

Example 3: Forward Fill (`ffill`)

Forward fill means:

> **Copy the previous row's value into the missing cell.**

print(df.fillna(method="ffill"))

Original DataFrame

Row	Name	Age	City

0	Alice	25	NYC
1	Bob	NaN	LA
2	None	30	Chicago

Output

Row	Name	Age	City
0	Alice	25	NYC
1	Bob	25	LA
2	Bob	30	Chicago

Explanation

Row 1

Age is missing.

Previous row's Age = **25**

So,

NaN → 25

Row 2

Name is missing.

Previous row's Name = **Bob**

So,

None → Bob

Forward fill always copies **the value from the row above**.

(Extra) Backward Fill (`bfill`)

Backward fill works in the opposite direction.

It copies the **next row's value** into the missing cell.

Example:

Nam e

Alic
e

Non
e

Bob

After `bfill()`:

**Nam
e**

Alic
e

Bob

Bob

4. Interpolating Missing Values

Interpolation means:

> **Estimate the missing numeric value using nearby values.**

Method:

interpolate()

This is mainly used for **numeric data.**

Example

```
data = pd.DataFrame({
    "Score":[100,None,80,None,60]
})

print(data.interpolate())
```

Original Data

Score
100
NaN
80

NaN

60

Output

Score
100
90
80
70
60

Explanation

For the first missing value:

Between **100** and **80**

Average

(100 + 80) / 2 = 90

So,

NaN → 90

Second missing value

Between **80** and **60**

Average

(80 + 60) / 2 = 70

So,

NaN → 70

This method is called **Linear Interpolation**, where Pandas estimates missing values by using the values before and after them.

5. Counting Missing Values

Sometimes we only want to know **how many missing values are present in each column**.

Use:

df.isna().sum()

Example

print(df.isna().sum())

Output

Name 1

Age 1

City 0

dtype: int64

Explanation

Pandas first checks every cell using `isna()`:

- Missing value → `True`
- Available value → `False`

Then `sum()` counts the number of `True` values in each column.

Result:

- **Name** → 1 missing value
- **Age** → 1 missing value
- **City** → 0 missing values

Summary (Exam Revision)

- **Missing data** means some values are empty or unavailable.
- Pandas represents missing values as `NaN` (numeric data) and `None` (object/string data).
- `isna()` / `isnull()` → Find missing values.
- `notna()` / `notnull()` → Find available values.
- `dropna()` → Remove rows or columns with missing values.
- `fillna()` → Replace missing values with a constant, different values for each column, or by using forward/backward fill.
- **Forward Fill (`ffill`)** → Copies the previous row's value into the missing cell.
- **Backward Fill (`bfill`)** → Copies the next row's value into the missing cell.
- `interpolate()` → Estimates missing numeric values using nearby values (linear interpolation by default).
- `isna().sum()` → Counts the number of missing values in each column.

7Q) How to Combine datasets in Pandas.

Ans:

Combining Datasets in Pandas

Definition (1 line)

Combining datasets means joining two or more DataFrames or Series into a single DataFrame.

We use combining when data is stored in different tables but we want them together.

Why do we combine datasets?

Suppose one file contains marks of one class and another file contains marks of another class.

DataFrame 1

Name	Marks
Alice	90
Bob	85

DataFrame 2

Name	Marks
Charlie	88
David	92

Instead of working with two separate DataFrames, we can combine them into one.

Methods to Combine Datasets

There are two common methods:

1. `concat()` (Recommended)
2. `append()` (Older method – Deprecated)

1. `concat()` Method

Definition

`concat()` is used to combine two or more DataFrames either row-wise or column-wise.

Syntax

pd.concat([df1, df2], axis=0)

or

pd.concat([df1, df2], axis=1)

Understanding the `axis` parameter

- `axis=0` → Combine **rows** (Vertical)

- `axis=1` → Combine **columns** (Horizontal)

Example 1: Concatenate Rows (Vertical)

```
import pandas as pd

df1 = pd.DataFrame({
    "Name":["Alice","Bob"],
    "Marks":[90,85]
})

df2 = pd.DataFrame({
    "Name":["Charlie","David"],
    "Marks":[88,92]
})

result = pd.concat([df1, df2])

print(result)
```

Output

Index	Name	Marks
0	Alice	90
1	Bob	85
0	Charlie	88
1	David	92

Explanation

Pandas places **df2 below df1**.

Think of stacking books one above another.

df1

Alice

Bob

↓

df2

Charlie

David

Final DataFrame

Alice

Bob

Charlie

David

Notice that the index is repeated because Pandas keeps the original indexes.

Ignore Old Index

Use

pd.concat([df1, df2], ignore_index=True)

Output

Index	Name	Marks
0	Alice	90
1	Bob	85
2	Charlie	88

3 David 92

Explanation

`ignore_index=True`

creates a new continuous index.

Example 2: Concatenate Columns (Horizontal)

```
df1 = pd.DataFrame({
    "Name":["Alice","Bob"]
})

df2 = pd.DataFrame({
    "Marks":[90,85]
})

print(pd.concat([df1, df2], axis=1))
```

Output

Name	Marks
Alice	90
Bob	85

Explanation

Since `axis=1`, Pandas joins columns side by side.

df1 df2

Name Marks

↓

Name Marks

Summary of `concat()`

Parameter	Meaning
`axis=0`	Join rows (Vertical)

`axis=1`	Join columns (Horizontal)
`ignore_index=True`	Creates a new index

2. `append()` Method

Definition

`append()` was used to add one DataFrame to another by rows.

Syntax

df1.append(df2)

Example

```
df1 = pd.DataFrame({
    "Name":["Alice","Bob"]
})

df2 = pd.DataFrame({
    "Name":["Charlie","David"]
})
print(df1.append(df2))
```

Output

Index	Name
0	Alice
1	Bob
0	Charlie
1	David

Explanation

`append()` simply adds **df2 below df1**, just like `concat(axis=0)`.

Why is `append()` not recommended?

From **Pandas version 2.0**, the `append()` method has been **removed (deprecated)**.

If you try to use:

df1.append(df2)

you will get an error.

Instead, always use:

pd.concat([df1, df2])

So, `concat()` replaces `append()`.

Difference Between `concat()` and `append()`

`concat()`	`append()`
Combines two or more DataFrames	Adds one DataFrame to another
Can join rows (`axis=0`) or columns (`axis=1`)	Joins only rows
Faster and more flexible	Less flexible
Recommended in modern Pandas	Deprecated and removed in Pandas 2.0

Exam Definition (2 Marks)

Combining datasets in Pandas is the process of joining two or more DataFrames or Series into a single DataFrame. This can be done using `concat()`, which supports both row-wise and column-wise combination.

The older `append()` method was used only for adding rows but is now deprecated.

Quick Revision

- **Combining datasets** → Join multiple DataFrames into one.
- **`pd.concat()`** → Recommended method.
- **`axis=0`** → Combine rows (Vertical).
- **`axis=1`** → Combine columns (Horizontal).
- **`ignore_index=True`** → Creates a new sequential index.
- **`append()`** → Older method for row-wise combination; **deprecated in Pandas 2.0**, so use `concat()` instead.

8Q) What is hierarchical indexing in Pandas? Explain how multi-level indices are created and accessed with examples.

Ans:

Definition

Hierarchical indexing, also called Multi-Level Indexing, is a feature in Pandas that allows a DataFrame or Series to have more than one index level. It helps organize and analyze complex data efficiently.

In a normal DataFrame, there is only **one index**.

Example:

Index	Name	Marks
0	Alice	90
1	Bob	85
2	Charlie	88

Here, the DataFrame has **one index level** (0, 1, 2).

In hierarchical indexing, there are **multiple index levels**.

Example:

State	City	Population
Andhra Pradesh	Vijayawada	12,00,000
Andhra Pradesh	Visakhapatnam	20,00,000
Telangana	Hyderabad	70,00,000

Here,

- **Level 1 Index = State**

- **Level 2 Index = City**

This is called **Hierarchical (Multi-Level) Indexing**.

Why Do We Need Hierarchical Indexing?

In real-world datasets, data is often grouped into multiple categories.

Examples:

- Country → State → City
- Department → Employee
- Year → Month
- Student → Subject

Instead of creating many separate DataFrames, we can organize all the data in one DataFrame using multiple index levels.

Advantages

- Organizes complex data neatly.
- Makes data selection easier.
- Supports grouping and summarization.
- Saves memory by avoiding duplicate labels.
- Makes data analysis more efficient.

Creating Multi-Level Indices

There are several ways to create hierarchical indexing.

Method 1: Using `set_index()`

This is the simplest and most common method.

Example

```
import pandas as pd


data = {
    "State": ["AP", "AP", "TS", "TS"],
    "City": ["Vijayawada", "Visakhapatnam", "Hyderabad", "Warangal"],
    "Population": [1200000, 2000000, 7000000, 800000]
}


df = pd.DataFrame(data)


df = df.set_index(["State", "City"])


print(df)
```

Output

State	City	Population

AP	Vijayawada	1200000
AP	Visakhapatnam	2000000
TS	Hyderabad	7000000
TS	Warangal	800000

Explanation

`set_index(["State","City"])`

creates two index levels:

- Level 1 → State
- Level 2 → City

Now both columns become the index.

Method 2: Using `MultiIndex.from_arrays()`

Example

import pandas as pd

index = pd.MultiIndex.from_arrays(

[["AP","AP","TS","TS"],

```
    ["Vijayawada","Visakhapatnam","Hyderabad","Warangal"]]
)

df = pd.DataFrame(
    {"Population":[1200000,2000000,7000000,800000]},
    index=index
)

print(df)
```

Explanation

Here,

`from_arrays()` creates multiple index levels using arrays.

Method 3: Using `MultiIndex.from_tuples()`

Example

```
import pandas as pd

index = pd.MultiIndex.from_tuples([
```

```
    ("AP","Vijayawada"),
    ("AP","Visakhapatnam"),
    ("TS","Hyderabad"),
    ("TS","Warangal")
])

df = pd.DataFrame(
    {"Population":[1200000,2000000,7000000,800000]},
    index=index
)

print(df)
```

Explanation

Each tuple represents one row with two index levels.

Method 4: Using `MultiIndex.from_product()`

Creates all possible combinations.

Example

```
import pandas as pd


index = pd.MultiIndex.from_product(
    [["AP","TS"],
     ["Vijayawada","Hyderabad"]]
)


print(index)
```

Output

(AP, Vijayawada)

(AP, Hyderabad)

(TS, Vijayawada)

(TS, Hyderabad)

Explanation

It creates every possible combination of the given lists.

Accessing Data in Hierarchical Indexing

Pandas provides several ways to access multi-level indexed data.

1. Access Using `.loc[]`

Example

```
df.loc["AP"]
```

Output

City	Population
Vijayawada	1200000
Visakhapatnam	2000000

Explanation

Returns all cities belonging to **AP**.

2. Access a Specific Row

```
df.loc[("AP","Vijayawada")]
```

Output

Population 1200000

Explanation

Returns data for one specific city.

3. Access Using `.xs()` (Cross Section)

df.xs("AP")

Output

City	Population
Vijayawada	1200000
Visakhapatnam	2000000

Explanation

`xs()` selects all rows belonging to one index level.

4. Selecting an Entire Second-Level Index

df.xs("Hyderabad", level="City")

Output

State	Population
TS	7000000

Explanation

Returns every row where City = Hyderabad.

Resetting the Index

To convert a MultiIndex back into normal columns:

df.reset_index()

Output

State	City	Population
AP	Vijayawada	1200000
AP	Visakhapatnam	2000000

TS Hyderabad 7000000

TS Warangal 800000

Explanation

`reset_index()` removes the hierarchical index and restores the index columns as normal DataFrame columns.

Sorting a Multi-Level Index

df.sort_index()

Explanation

Sorts the DataFrame based on all index levels.

Example of Hierarchical Indexing in Student Data

import pandas as pd

data = {

"Class":["CSE","CSE","ECE","ECE"],

```
    "Student":["Alice","Bob","Charlie","David"],
    "Marks":[90,85,88,92]
}

df = pd.DataFrame(data)

df = df.set_index(["Class","Student"])

print(df)
```

Output

Class	Student	Marks
CSE	Alice	90
CSE	Bob	85
ECE	Charlie	88

EC David 92
E

Access Examples

df.loc["CSE"]

Returns all CSE students.

df.loc[("ECE","David")]

Returns David's marks.

Advantages of Hierarchical Indexing

- Organizes large and complex datasets efficiently.
- Supports multiple index levels.
- Makes data selection and filtering easier.
- Improves grouping and aggregation.
- Useful for time-series and business data.
- Reduces duplicate labels.
- Simplifies data analysis.

Limitations

- Slightly harder for beginners to understand.
- Selecting data requires more care than a normal index.

- May become confusing with many index levels.

Applications

Hierarchical indexing is commonly used in:

- Sales reports (Year → Month → Day)
- Student records (Class → Student)
- Company data (Department → Employee)
- Geography (Country → State → City)
- Financial and stock market data
- Weather and time-series datasets

Difference Between Single Index and Hierarchical Index

Single Index	Hierarchical Index
One index level	Two or more index levels
Simple structure	Multi-level structure
Suitable for small datasets	Suitable for large and complex datasets
Easier to understand	More powerful for organizing grouped data

Conclusion

Hierarchical indexing (Multi-Level Indexing) in Pandas allows a DataFrame or Series to use two or more index levels. It helps organize complex datasets, makes data selection and grouping easier, and is widely used in real-world applications such as sales analysis, student records, geographical data, and time-series analysis. It can be created using `set_index()` or `MultiIndex` methods and accessed using functions like `loc()` and `xs()`.

Exam Tips (20 Marks)

- Start with the definition.
- Explain why hierarchical indexing is needed.
- Describe at least **3–4 methods** of creating a MultiIndex (`set_index()`, `from_arrays()`, `from_tuples()`, `from_product()`).
- Explain how to access data using `loc[]` and `xs()`.
- Include `reset_index()` and `sort_index()`.
- Mention advantages, limitations, applications, and conclude with a comparison to a single index. This level of detail is suitable for a **20-mark university answer**.

9Q) Create a pandas data frame using lists of tuples.

Ans:

Definition

A DataFrame can be created from a list of tuples, where each tuple represents one row of data.

Syntax

```
import pandas as pd

data = [

    (value1, value2, value3),

    (value1, value2, value3),

    ...

]


df = pd.DataFrame(data, columns=["Column1", "Column2", "Column3"])
```

- `data` → List of tuples.
- Each **tuple** represents one row.
- `columns` → Specifies the column names.

Example

```
import pandas as pd


data = [
    ("Alice", 21, "CSE"),
    ("Bob", 20, "ECE"),
    ("Charlie", 22, "ME")
]


df = pd.DataFrame(data, columns=["Name", "Age", "Branch"])


print(df)
```

Output

Name	Age	Branch
Alice	21	CSE
Bob	20	ECE

Charl 22 ME
ie

Explanation

- A list named `data` contains **three tuples**.
- Each tuple represents **one student's details.**
- `pd.DataFrame()` converts the list of tuples into a DataFrame.
- The `columns` parameter assigns the column names: **Name, Age, and Branch**.

Another Example

import pandas as pd

employees = [

(101, "Rahul", 45000),

(102, "Priya", 50000),

(103, "Kiran", 48000)

]

df = pd.DataFrame(employees, columns=["Emp_ID", "Name", "Salary"])

```
print(df)
```

Output

Emp_ID	Name	Salary
101	Rahul	45000
102	Priya	50000
103	Kiran	48000

Exam Definition (2 Marks)

A Pandas DataFrame can be created using a list of tuples by passing the list to the `pd.DataFrame()` function and specifying the column names using the `columns` parameter. Each tuple represents one row in the DataFrame.

Key Points to Remember

- A **list** contains multiple tuples.
- Each **tuple** becomes one row.
- Use `pd.DataFrame()` to create the DataFrame.

- Use the `columns` parameter to specify column names.

10Q) Explain the role of pandas in the Python for data analysis. How does it complement other libraries like NumPy in handling structured data?

Ans:

Introduction

Pandas is one of the most important Python libraries used for data analysis and data manipulation. It provides powerful data structures such as Series and DataFrame, which make it easy to store, organize, clean, analyze, and manipulate structured data.

Pandas is widely used in:

- Data Science
- Machine Learning
- Data Analytics
- Business Intelligence
- Financial Analysis

What is Pandas?

Definition (1 line):

Pandas is a Python library used to organize, manipulate, analyze, and process structured data efficiently.

Role of Pandas in Data Analysis

Pandas plays a major role in every step of the data analysis process.

1. Data Loading (Importing Data)

Pandas can read data from different file formats.

Examples:

- CSV files
- Excel files
- SQL databases
- JSON files

Example:

import pandas as pd

df = pd.read_csv("students.csv")

Role:

- Reads data quickly.
- Converts the file into a DataFrame for analysis.

2. Data Organization

Pandas stores data in a tabular format using a **DataFrame**.

Example

Name	Age	Marks
Alice	20	90
Bob	21	85

This makes the data easy to understand and work with.

3. Data Cleaning

Real-world data often contains:

- Missing values
- Duplicate records
- Incorrect values

Pandas provides functions like:

- `dropna()`
- `fillna()`
- `drop_duplicates()`
- `replace()`

These functions help clean the dataset before analysis.

4. Data Selection and Filtering

Pandas allows us to select:

- Rows
- Columns
- Specific values
- Data based on conditions

Example

df[df["Marks"] > 80]

This selects students whose marks are greater than 80.

5. Data Manipulation

Pandas can:

- Add new columns
- Delete columns
- Rename columns
- Sort data
- Combine datasets
- Merge tables

This makes data modification very simple.

6. Statistical Analysis

Pandas provides built-in statistical functions.

Examples

df.mean()

df.max()

df.min()

df.sum()

df.describe()

These functions quickly calculate statistical information.

7. Grouping Data

Pandas can group similar data together using `groupby()`.

Example

Find the average salary department-wise.

df.groupby("Department").mean()

This is useful in business reports.

8. Time Series Analysis

Pandas provides excellent support for:

- Dates
- Time
- Monthly reports
- Yearly reports

This is useful in:

- Stock market analysis
- Weather forecasting
- Sales analysis

9. Data Visualization Support

Pandas works well with visualization libraries like:

- Matplotlib
- Seaborn

Example

df.plot()

It allows us to create graphs directly from a DataFrame.

How Pandas Complements NumPy

Pandas is built **on top of NumPy**.

This means Pandas uses NumPy internally for fast calculations.

Think of it like this:

- **NumPy is the engine.**
- **Pandas is the car built using that engine.**

NumPy provides speed, while Pandas provides an easy way to work with structured data.

Difference Between NumPy and Pandas

NumPy	Pandas
Works with arrays	Works with Series and DataFrames
Mainly numerical data	Numerical and non-numerical data
No row or column labels	Supports row and column labels
Faster for mathematical calculations	Easier for data analysis
Less flexible for tabular data	Excellent for structured/tabular data

How Pandas Uses NumPy

1. Storage

Pandas stores data internally using NumPy arrays.

So Pandas gets NumPy's speed automatically.

2. Mathematical Operations

Pandas uses NumPy functions for fast calculations.

Example

df["Marks"].mean()

Internally, NumPy performs the calculation.

3. Missing Values

NumPy provides:

NaN

Pandas uses this to represent missing numerical values.

4. Vectorized Operations

Instead of using loops, Pandas performs operations on entire columns using NumPy.

Example

df["Marks"] = df["Marks"] + 5

Every value increases by 5 at once.

This is much faster than using a loop.

Why Use Pandas Instead of Only NumPy?

Suppose we have student data.

Using NumPy

[90,85,95]

We know only the marks.

But with Pandas

Name	Marks
Alice	90

Bob 85

Charlie 95

Now we know:

- Student names
- Marks
- Labels
- Easy filtering
- Easy searching

This is why Pandas is more suitable for structured data.

Advantages of Pandas

- Easy to learn and use.
- Handles large datasets efficiently.
- Supports missing values.
- Provides powerful data analysis tools.
- Reads and writes multiple file formats.
- Integrates well with NumPy, Matplotlib, and Seaborn.
- Supports grouping, sorting, filtering, and merging.
- Widely used in Data Science and Machine Learning.

Applications of Pandas

Pandas is used in:

- Data Science
- Machine Learning
- Business Analytics
- Financial Analysis
- Healthcare Data Analysis
- Sales and Marketing Reports
- Stock Market Analysis
- Scientific Research

Conclusion

Pandas is a powerful Python library for data analysis and manipulation. It simplifies working with structured data by providing easy-to-use data structures like Series and DataFrame. Pandas complements NumPy by combining NumPy's fast numerical computations with labeled data structures, making tasks such as data cleaning, filtering, grouping, statistical analysis, and visualization much easier. Together, NumPy and Pandas form the foundation of Python-based data analysis.

Exam Tips (20 Marks)

To score full marks:

1. Write the definition of Pandas.
2. Explain the role of Pandas in data analysis (loading, cleaning, organizing, filtering, manipulation, statistics, grouping, visualization).
3. Explain how Pandas complements NumPy.
4. Include a comparison table between Pandas and NumPy.

5. Mention the advantages, applications, and conclude with the importance of using both libraries together.

PANDAS PRACTICE PROGRAMS

Topics: Indexing, Selection, Operating on Data in Pandas, Handling Missing Data, Combining Datasets (Concat & Append)

1 INDEXING – BASICS

Create a DataFrame df with the following data:
Columns: ['ID', 'Name', 'Age', 'City']
Rows:
(1, 'Alice', 25, 'New York')
(2, 'Bob', 30, 'Los Angeles')
(3, 'Charlie', 35, 'Chicago')
(4, 'David', 28, 'Houston')

Tasks:
1. Set 'ID' as the index of the DataFrame.
2. Access the row with ID = 3.
3. Access the 'Name' of the person with ID = 2.
4. Reset the index of the DataFrame.

2 SELECTION – SINGLE & MULTIPLE

Given a DataFrame df with columns:
['Product', 'Category', 'Price', 'Quantity', 'Sales']
and 10 rows of sample data.

Tasks:
1. Select the 'Product' column.
2. Select the first 3 rows.
3. Select rows where 'Category' is 'Electronics'.
4. Select rows where 'Price' > 500.
5. Select columns 'Product' and 'Sales'.
6. Select rows where 'Quantity' >= 10 and 'Sales' > 1000.

3 SELECTION – LOC & ILOC

Given a DataFrame df with 8 rows and 5 columns
(columns: ['A', 'B', 'C', 'D', 'E']).

Tasks:
1. Select the row at index label 3.
2. Select rows with index labels 2 to 5 (inclusive).
3. Select the element at row label 4 and column 'C' using loc.
4. Select the first 3 rows using iloc.
5. Select all rows and columns at positions [1, 3, 4] using iloc.
6. Select the element at row position 2 and column position 3.

4 OPERATING ON DATA – BASICS

Given a DataFrame df with columns:
['Item', 'Price', 'Quantity', 'Discount']
and 10 rows of sample data.

Tasks:
1. Create a new column 'Total' = Price * Quantity.
2. Create a new column 'Discounted_Total' = Total – Discount.
3. Create a new column 'Tax' = 0.18 * Discounted_Total.
4. Create a new column 'Final_Amount' = Discounted_Total + Tax.
5. Round 'Final_Amount' to 2 decimal places.

5 OPERATING ON DATA – AGGREGATIONS

Given a DataFrame sales with columns:
['Region', 'Product', 'Sales', 'Quantity']
and multiple rows.

Tasks:
1. Find the total sales.
2. Find the average sales per product.
3. Find the total quantity sold per region.
4. Find the maximum sale value per region.
5. Group by 'Region' and find total sales and average sales.
6. Group by 'Product' and find total quantity sold.

6 HANDLING MISSING DATA – DETECTION

Given a DataFrame df with missing values (NaN) in various columns.

Tasks:
1. Identify columns with any missing values.
2. Count total missing values in each column.
3. Count total missing values in the entire DataFrame.
4. Select rows that have at least one missing value.
5. Select rows that have no missing values.

7 HANDLING MISSING DATA – IMPUTATION

Given a DataFrame df with numerical columns
['Age', 'Salary', 'Bonus'] and categorical columns
['Department', 'City'] containing missing values.

Tasks:
1. Fill missing values in numerical columns with the mean.
2. Fill missing values in numerical columns with the median.
3. Fill missing values in categorical columns with the mode.
4. Fill missing values in 'Department' with 'Unknown'.
5. Interpolate missing values in numerical columns.

8 HANDLING MISSING DATA – DROP

Given a DataFrame df with missing values.

Tasks:
1. Drop rows with any missing values.
2. Drop columns with any missing values.
3. Drop rows where 'Salary' is missing.
4. Drop rows where more than 2 columns are missing.
5. Drop columns where more than 30% values are missing.

9 COMBINING DATASETS – CONCATENATE

Given three DataFrames:
df1, df2, df3 (each with the same columns).

Tasks:
1. Concatenate df1, df2, df3 vertically (row-wise).
2. Concatenate df1 and df2 horizontally (column-wise).
3. Concatenate all three with keys ['DF1', 'DF2', 'DF3'] as hierarchical index.
4. Concatenate with ignore_index=True.

10 COMBINING DATASETS – APPEND

Given two DataFrames df1 and df2 with the same columns.

Tasks:
1. Append df2 to df1 using the append() method.
2. Append df2 to df1 and reset the index.
3. Append multiple DataFrames in a list to df1.
4. Add a column 'Source' to identify from which DataFrame each row came before appending.

11 REAL WORLD – CLEAN AND ANALYZE

Given a messy sales DataFrame df with missing values, duplicate rows, and inconsistent data.

Tasks:
1. Handle missing values appropriately.
2. Remove duplicate rows.
3. Standardize text in 'Product' column (strip & title case).
4. Create necessary new columns (e.g., 'Total_Sales').
5. Perform summary statistics (total, mean, count by category).

12 COMPREHENSIVE PRACTICE

Given three DataFrames:
df_customers, df_orders, df_products.

Tasks:
1. Clean missing data in all DataFrames.
2. Merge df_orders with df_customers on 'CustomerID'.
3. Merge the result with df_products on 'ProductID'.
4. Perform groupby analysis: total sales per customer and per product category.
5. Export the final result to a CSV file.

```
import pandas as pd
import numpy as np

np.random.seed(42)

SEP = "=" * 65

#
————————————————————————————————————————————————————————————

# 1. INDEXING – BASICS
#
————————————————————————————————————————————————————————————

print(SEP)
print("1. INDEXING – BASICS")
print(SEP)

df = pd.DataFrame({
```

```
    'ID':   [1, 2, 3, 4],
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age':  [25, 30, 35, 28],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
})

print("Original DataFrame:")
print(df)

# Task 1: Set 'ID' as index
df.set_index('ID', inplace=True)
print("\nTask 1 – Set 'ID' as index:")
print(df)

# Task 2: Access row with ID = 3
print("\nTask 2 – Row with ID = 3:")
print(df.loc[3])
```

```
# Task 3: Name of person with ID = 2
print("\nTask 3 – Name of person with ID = 2:")
print(df.loc[2, 'Name'])

# Task 4: Reset index
df_reset = df.reset_index()
print("\nTask 4 – Reset index:")
print(df_reset)



#
————————————————————————————————————————————————————————————

# 2. SELECTION – SINGLE & MULTIPLE
#
————————————————————————————————————————————————————————————

print("\n" + SEP)
print("2. SELECTION – SINGLE & MULTIPLE")
```

```
print(SEP)


df = pd.DataFrame({
                                                    'Product':
['Laptop','Phone','Tablet','Monitor','Keyboard','Mouse','Speaker
','Webcam','Printer','Desk'],
                                                    'Category':
['Electronics','Electronics','Electronics','Electronics','Accessorie
s',


'Accessories','Electronics','Accessories','Electronics','Furniture'
],
    'Price':    [999, 699, 450, 350, 120, 45, 200, 80, 600,
250],
   'Quantity': [10, 25, 15, 8, 50, 100, 20, 30, 5, 3],
    'Sales':    [9990, 17475, 6750, 2800, 6000, 4500, 4000,
2400, 3000, 750]
})


print("DataFrame:")
```

```
print(df)

# Task 1
print("\nTask 1 – 'Product' column:")
print(df['Product'])

# Task 2
print("\nTask 2 – First 3 rows:")
print(df.head(3))

# Task 3
print("\nTask 3 – Category == 'Electronics':")
print(df[df['Category'] == 'Electronics'])

# Task 4
print("\nTask 4 – Price > 500:")
print(df[df['Price'] > 500])
```

```
# Task 5
print("\nTask 5 – Columns 'Product' and 'Sales':")
print(df[['Product', 'Sales']])

# Task 6
print("\nTask 6 – Quantity >= 10 AND Sales > 1000:")
print(df[(df['Quantity'] >= 10) & (df['Sales'] > 1000)])



#
————————————————————————————————————————————————————————————

# 3. SELECTION – LOC & ILOC
#
————————————————————————————————————————————————————————————

print("\n" + SEP)
print("3. SELECTION – LOC & ILOC")
print(SEP)
```

```
df = pd.DataFrame(
    np.random.randint(10, 99, size=(8, 5)),
    columns=['A', 'B', 'C', 'D', 'E']
)
print("DataFrame:")
print(df)

# Task 1
print("\nTask 1 – Row at index label 3:")
print(df.loc[3])

# Task 2
print("\nTask 2 – Rows with index labels 2 to 5 (inclusive):")
print(df.loc[2:5])

# Task 3
print("\nTask 3 – Element at row label 4, column 'C' (loc):")
```

```
print(df.loc[4, 'C'])


# Task 4

print("\nTask 4 – First 3 rows (iloc):")

print(df.iloc[:3])


# Task 5

print("\nTask 5 – All rows, columns at positions [1, 3, 4] (iloc):")

print(df.iloc[:, [1, 3, 4]])


# Task 6

print("\nTask 6 – Element at row position 2, column position 3 (iloc):")

print(df.iloc[2, 3])
```

```
#
-------------------------------------------------------------
-------------------------------------------

# 4. OPERATING ON DATA – BASICS

#
-------------------------------------------------------------
-------------------------------------------

print("\n" + SEP)

print("4. OPERATING ON DATA – BASICS")

print(SEP)


df = pd.DataFrame({

    'Item':     ['A','B','C','D','E','F','G','H','I','J'],

      'Price':      [100, 250, 80, 400, 150, 320, 90, 200, 130,
175],

    'Quantity': [5, 2, 10, 1, 7, 3, 8, 4, 6, 9],

    'Discount': [20, 50, 10, 80, 30, 60, 15, 40, 25, 35]

})

print("Original DataFrame:")
```

```
print(df)


# Task 1
df['Total'] = df['Price'] * df['Quantity']
# Task 2
df['Discounted_Total'] = df['Total'] - df['Discount']
# Task 3
df['Tax'] = 0.18 * df['Discounted_Total']
# Task 4
df['Final_Amount'] = df['Discounted_Total'] + df['Tax']
# Task 5
df['Final_Amount'] = df['Final_Amount'].round(2)


print("\nDataFrame after all operations:")
print(df)
```

```
#
──────────────────────────────────────────────────────────────────────
──────────────────────────────────────────

# 5. OPERATING ON DATA – AGGREGATIONS
#
──────────────────────────────────────────────────────────────────────
──────────────────────────────────────────

print("\n" + SEP)
print("5. OPERATING ON DATA – AGGREGATIONS")
print(SEP)


sales = pd.DataFrame({
                                                    'Region':
['North','South','East','West','North','South','East','West','Nort
h','East'],
                                                    'Product':
['Laptop','Phone','Tablet','Laptop','Phone','Tablet','Laptop','Ph
one','Tablet','Phone'],
    'Sales':    [5000, 3000, 2000, 4500, 2800, 1500, 3200,
2100, 1800, 2500],
```

```
    'Quantity': [10, 15, 8, 9, 14, 6, 8, 11, 7, 12]
})
print("Sales DataFrame:")
print(sales)

print(f"\nTask 1 – Total Sales: {sales['Sales'].sum()}")
print(f"\nTask 2 – Average Sales per Product:")
print(sales.groupby('Product')['Sales'].mean())
print(f"\nTask 3 – Total Quantity sold per Region:")
print(sales.groupby('Region')['Quantity'].sum())
print(f"\nTask 4 – Maximum Sale value per Region:")
print(sales.groupby('Region')['Sales'].max())
print(f"\nTask 5 – Total & Average Sales by Region:")
print(sales.groupby('Region')['Sales'].agg(['sum', 'mean']))
print(f"\nTask 6 – Total Quantity by Product:")
print(sales.groupby('Product')['Quantity'].sum())
```

```
#
-------------------------------------------------------------
-------------------------------------------

# 6. HANDLING MISSING DATA – DETECTION
#
-------------------------------------------------------------
-------------------------------------------

print("\n" + SEP)
print("6. HANDLING MISSING DATA – DETECTION")
print(SEP)


df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', None, 'David', 'Eve'],
    'Age':    [25, np.nan, 35, np.nan, 29],
    'Salary': [50000, 60000, np.nan, 70000, np.nan],
    'City':   ['NY', 'LA', 'Chicago', None, 'Houston'],
    'Score':  [88, np.nan, 72, 95, np.nan]
})
print("DataFrame with missing values:")
```

```
print(df)


# Task 1
print("\nTask 1 – Columns with any missing values:")
print(df.columns[df.isnull().any()].tolist())


# Task 2
print("\nTask 2 – Missing value count per column:")
print(df.isnull().sum())


# Task 3
print(f"\nTask 3 – Total missing values in DataFrame: {df.isnull().sum().sum()}")


# Task 4
print("\nTask 4 – Rows with at least one missing value:")
print(df[df.isnull().any(axis=1)])
```

```
# Task 5
print("\nTask 5 – Rows with NO missing values:")
print(df[df.notnull().all(axis=1)])



#
____________________________________________________________
____________________________________________

# 7. HANDLING MISSING DATA – IMPUTATION
#
____________________________________________________________
____________________________________________

print("\n" + SEP)
print("7. HANDLING MISSING DATA – IMPUTATION")
print(SEP)

df = pd.DataFrame({
    'Age':       [25, np.nan, 35, 28, np.nan, 40, 32, np.nan],
```

```
    'Salary':     [50000, 60000, np.nan, 70000, 55000, np.nan, 65000, 72000],
    'Bonus':      [2000, np.nan, 3500, np.nan, 2500, 4000, np.nan, 3000],
    'Department': ['HR', 'IT', np.nan, 'Finance', 'IT', np.nan, 'HR', 'Finance'],
    'City':       ['NY', 'LA', 'Chicago', np.nan, 'LA', 'NY', np.nan, 'Chicago']
})
print("Original DataFrame:")
print(df)

df2 = df.copy()

# Task 1: Fill numerical with mean
for col in ['Age', 'Salary', 'Bonus']:
    df2[col] = df2[col].fillna(df2[col].mean())
print("\nTask 1 – Numerical filled with mean:")
print(df2[['Age', 'Salary', 'Bonus']].round(2))
```

```
# Task 2: Fill numerical with median (on fresh copy)
df3 = df.copy()
for col in ['Age', 'Salary', 'Bonus']:
    df3[col] = df3[col].fillna(df3[col].median())
print("\nTask 2 – Numerical filled with median:")
print(df3[['Age', 'Salary', 'Bonus']].round(2))

# Task 3: Fill categorical with mode
df2['City'] = df2['City'].fillna(df2['City'].mode()[0])
print("\nTask 3 – 'City' filled with mode:")
print(df2['City'])

# Task 4: Fill Department with 'Unknown'
df2['Department'] = df2['Department'].fillna('Unknown')
print("\nTask 4 – 'Department' filled with 'Unknown':")
print(df2['Department'])
```

```
# Task 5: Interpolate
df4 = df.copy()
for col in ['Age', 'Salary', 'Bonus']:
    df4[col] = df4[col].interpolate()
print("\nTask 5 – Numerical columns interpolated:")
print(df4[['Age', 'Salary', 'Bonus']].round(2))



#
————————————————————————————————————————————————————————————

# 8. HANDLING MISSING DATA – DROP
#
————————————————————————————————————————————————————————————

print("\n" + SEP)
print("8. HANDLING MISSING DATA – DROP")
print(SEP)
```

```
df = pd.DataFrame({
    'Name':   ['Alice', 'Bob', None, 'David', 'Eve', 'Frank'],
    'Age':    [25, np.nan, 35, np.nan, 29, 33],
    'Salary': [50000, 60000, np.nan, 70000, np.nan, 58000],
    'City':   ['NY', 'LA', 'Chicago', None, 'Houston', 'NY'],
    'Score':  [88, np.nan, 72, 95, np.nan, 80]
})
print("Original DataFrame:")
print(df)
print(f"Shape: {df.shape}")

# Task 1
print(f"\nTask 1 – Drop rows with ANY missing:\n{df.dropna()}")

# Task 2
print(f"\nTask 2 – Drop columns with ANY missing:\n{df.dropna(axis=1)}")
```

```
# Task 3
print(f"\nTask 3 – Drop rows where 'Salary' is missing:\n{df[df['Salary'].notna()]}")

# Task 4
print(f"\nTask 4 – Drop rows where more than 2 cols are missing:\n{df.dropna(thresh=df.shape[1]-2)}")

# Task 5
threshold = int(0.30 * len(df))
cols_to_drop = df.columns[df.isnull().sum() > threshold].tolist()
print(f"\nTask 5 – Columns with >30% missing (dropped): {cols_to_drop}")
print(df.drop(columns=cols_to_drop))

#
```

```
# 9. COMBINING DATASETS – CONCATENATE
#
──────────────────────────────────────────────────────────────
──────────────────────────────────────────

print("\n" + SEP)
print("9. COMBINING DATASETS – CONCATENATE")
print(SEP)

df1 = pd.DataFrame({'ID':[1,2,3],
'Name':['Alice','Bob','Charlie'], 'Score':[85,90,78]})
df2 = pd.DataFrame({'ID':[4,5], 'Name':['David','Eve'],
'Score':[92,88]})
df3 = pd.DataFrame({'ID':[6,7], 'Name':['Frank','Grace'],
'Score':[76,95]})

print("df1:"); print(df1)
print("df2:"); print(df2)
print("df3:"); print(df3)
```

```
# Task 1: Vertical concat
print("\nTask 1 – Vertical concatenation (df1+df2+df3):")
print(pd.concat([df1, df2, df3]))

# Task 2: Horizontal concat
df1_h = pd.DataFrame({'ID':[1,2,3], 'Name':['Alice','Bob','Charlie']})
df2_h = pd.DataFrame({'Score':[85,90,78], 'City':['NY','LA','Chicago']})
print("\nTask 2 – Horizontal concatenation (column-wise):")
print(pd.concat([df1_h, df2_h], axis=1))

# Task 3: Keys as hierarchical index
print("\nTask 3 – Concat with keys ['DF1','DF2','DF3']:")
print(pd.concat([df1, df2, df3], keys=['DF1','DF2','DF3']))

# Task 4: ignore_index=True
print("\nTask 4 – Concat with ignore_index=True:")
print(pd.concat([df1, df2, df3], ignore_index=True))
```

```
#
_____
_____

# 10. COMBINING DATASETS – APPEND
#
_____
_____

print("\n" + SEP)
print("10. COMBINING DATASETS – APPEND (via pd.concat)")
print(SEP)

df1 = pd.DataFrame({'ID':[1,2,3], 'Name':['Alice','Bob','Charlie'], 'Score':[85,90,78]})
df2 = pd.DataFrame({'ID':[4,5], 'Name':['David','Eve'], 'Score':[92,88]})

print("df1:"); print(df1)
```

```
print("df2:"); print(df2)

# Task 1: Append df2 to df1
print("\nTask 1 – Append df2 to df1:")
print(pd.concat([df1, df2]))

# Task 2: Append and reset index
print("\nTask 2 – Append and reset index:")
print(pd.concat([df1, df2], ignore_index=True))

# Task 3: Append multiple DataFrames
df3 = pd.DataFrame({'ID':[6,7], 'Name':['Frank','Grace'], 'Score':[76,95]})
print("\nTask 3 – Append multiple DataFrames [df2, df3] to df1:")
print(pd.concat([df1, df2, df3], ignore_index=True))

# Task 4: Add 'Source' column before appending
df1_s = df1.copy(); df1_s['Source'] = 'df1'
```

```
df2_s = df2.copy(); df2_s['Source'] = 'df2'
print("\nTask 4 – With 'Source' column:")
print(pd.concat([df1_s, df2_s], ignore_index=True))



#
————————————————————————————————————————
————————————————————————————————————

# 11. REAL WORLD – CLEAN AND ANALYZE
#
————————————————————————————————————————
————————————————————————————————————

print("\n" + SEP)
print("11. REAL WORLD – CLEAN AND ANALYZE")
print(SEP)


df = pd.DataFrame({
    'Product':   ['laptop', 'PHONE', 'tablet', 'Laptop', 'phone',
'TABLET',
```

```
'monitor', 'Monitor', 'keyboard', 'laptop'],
'Category': ['Electronics','Electronics','Electronics','Electronics','Electronics',

'Electronics','Electronics','Electronics','Accessories','Electronics'],
    'Price':    [999, 699, np.nan, 1099, 649, 499, 350, np.nan, 120, 999],
   'Quantity': [10, 25, 15, 10, np.nan, 8, 12, 12, 50, 10],
    'Sales':     [np.nan, 17475, 7485, 10990, 16225, 3992, 4200, 4200, 6000, 9990]
})

print("Messy DataFrame:")
print(df)

# Task 1: Handle missing values
df['Price'] = df['Price'].fillna(df['Price'].median())
df['Quantity'] = df['Quantity'].fillna(df['Quantity'].median())
```

```
df['Sales'] = df['Sales'].fillna(df['Price'] * df['Quantity'])

print("\nTask 1 – After handling missing values:")

print(df)


# Task 2: Remove duplicate rows

df.drop_duplicates(inplace=True)

print(f"\nTask 2 – After removing duplicates (shape: {df.shape}):")

print(df)


# Task 3: Standardize 'Product' column

df['Product'] = df['Product'].str.strip().str.title()

print("\nTask 3 – After standardizing 'Product' (strip + title case):")

print(df['Product'])


# Task 4: Create 'Total_Sales' column

df['Total_Sales'] = (df['Price'] * df['Quantity']).round(2)

print("\nTask 4 – New 'Total_Sales' column:")
```

```
print(df[['Product', 'Price', 'Quantity', 'Total_Sales']])


# Task 5: Summary statistics by category
print("\nTask 5 – Summary statistics by Product:")
print(df.groupby('Product')['Total_Sales'].agg(['sum','mean','count']).round(2))



#
————————————————————————————————————————————————————————————

# 12. COMPREHENSIVE PRACTICE
#
————————————————————————————————————————————————————————————

print("\n" + SEP)
print("12. COMPREHENSIVE PRACTICE")
print(SEP)
```

```
df_customers = pd.DataFrame({
    'CustomerID': [1, 2, 3, 4, 5],
    'Name':       ['Alice', 'Bob', np.nan, 'David', 'Eve'],
    'City':       ['NY', 'LA', 'Chicago', np.nan, 'Houston'],
      'Segment':       ['Retail',  'Wholesale',  'Retail',  'Wholesale',
np.nan]
})

df_orders = pd.DataFrame({
    'OrderID':    [101, 102, 103, 104, 105, 106],
    'CustomerID': [1, 2, 1, 3, 5, np.nan],
    'ProductID':  ['P1', 'P2', 'P3', 'P1', 'P2', 'P3'],
    'Amount':     [500, np.nan, 750, 300, 450, 200]
})

df_products = pd.DataFrame({
    'ProductID': ['P1', 'P2', 'P3'],
    'ProductName': ['Laptop', 'Phone', 'Tablet'],
```

```
    'Category':    ['Electronics', 'Electronics', 'Electronics']
})

print("df_customers:"); print(df_customers)
print("\ndf_orders:");   print(df_orders)
print("\ndf_products:"); print(df_products)

# Task 1: Clean missing data
df_customers['Name']                                    =
df_customers['Name'].fillna('Unknown')
df_customers['City'] = df_customers['City'].fillna('Unknown')
df_customers['Segment']                                 =
df_customers['Segment'].fillna(df_customers['Segment'].mode()
[0])
df_orders['Amount']                                     =
df_orders['Amount'].fillna(df_orders['Amount'].median())
df_orders.dropna(subset=['CustomerID'], inplace=True)
df_orders['CustomerID']                                 =
df_orders['CustomerID'].astype(int)
```

```
print("\nTask 1 – After cleaning:")
print(df_customers)
print(df_orders)

# Task 2: Merge orders with customers
merged = pd.merge(df_orders, df_customers, on='CustomerID', how='left')
print("\nTask 2 – Orders merged with Customers:")
print(merged)

# Task 3: Merge with products
final = pd.merge(merged, df_products, on='ProductID', how='left')
print("\nTask 3 – Merged with Products:")
print(final)

# Task 4: Groupby analysis
print("\nTask 4a – Total Sales per Customer:")
```

```
print(final.groupby('Name')['Amount'].sum().reset_index(name
='Total_Sales'))


print("\nTask 4b – Total Sales per Product Category:")
print(final.groupby('Category')['Amount'].sum().reset_index(na
me='Total_Sales'))


# Task 5: Export to CSV
output_path                                                    =
'/mnt/user-data/outputs/comprehensive_result.csv'
final.to_csv(output_path, index=False)
print(f"\nTask 5 – Exported to CSV: {output_path}")
print("Preview:")
print(final)
```
